STUFX - STUDENT STUDY TRACKER

STUDENT NAME - KARUMANCHI BHAVANA

STUDENT ID - 2520030253

CLUSTER - 01

SECTION - 03

Under the Guidance of
Supervisor Name -

ABSTRACT

- Smart Student Productivity Tracker is a web-based application designed to help students manage their study activities efficiently.
- HelpS students monitor learning progress and productivity levels.
- The system provides visual insights such as dashboards and statistics.
- Developed using modern web technologies to ensure accessibility and ease of use.
- Provides a platform to track daily study hours, manage subjects, and organize tasks.
- Enables students to maintain consistent study habits and better time management.
- The system provides visual insights such as dashboards and statistics.
- Useful for students preparing for exams, assignments, and long-term learnin goals.

EXISTING SYSTEM

- Students usually track study progress using manual methods such as notebooks or planners.
- Lack of a centralized system to store study data and task lists.
- Difficult to monitor daily or weekly study hours accurately.
- No automatic system to analyze productivity or generate reports.
- Students may forget pending tasks or study schedules.
- Manual tracking leads to poor organization and inefficient time management.
- Limited motivation due to lack of progress visualization.
- Existing methods do not support digital reminders or structured task management

PROPOSED SYSTEM

- A digital platform that allows students to monitor and manage their study activities
- Provides dashboard displaying study statistics and progress information.
- Allows students to add subjects, tasks, and track study sessions.
- Includes a study timer to record study duration accurately.
- Provides reports and charts for productivity analysis.
- Helps students maintain organized schedules and study routines.
- Encourages self-discipline and goal-oriented learning.
- Designed with a simple user interface for easy navigation and accessibility.

SYSTEM ARCHITECTURE

- The system follows a three-layer architecture for efficient processing and data management.

Presentation Layer (Frontend)

- Provides the user interface for interaction.
- Developed using HTML, CSS, JavaScript, and React.
- Displays dashboard, task lists, and study reports.

Application Layer (Backend)

- Handles business logic and application processing.
- Developed using Java programming language.
- Processes user requests and manages study data.

Data Layer (Storage)

- Stores student study records, tasks, and progress data.
- Data can be stored in database or local storage.

System Flow

- User → Frontend Interface → Backend Processing → Data Storage → Response to User

KLH (DEEMED TO BE UNIVERSITY)

PROJECT MODULES

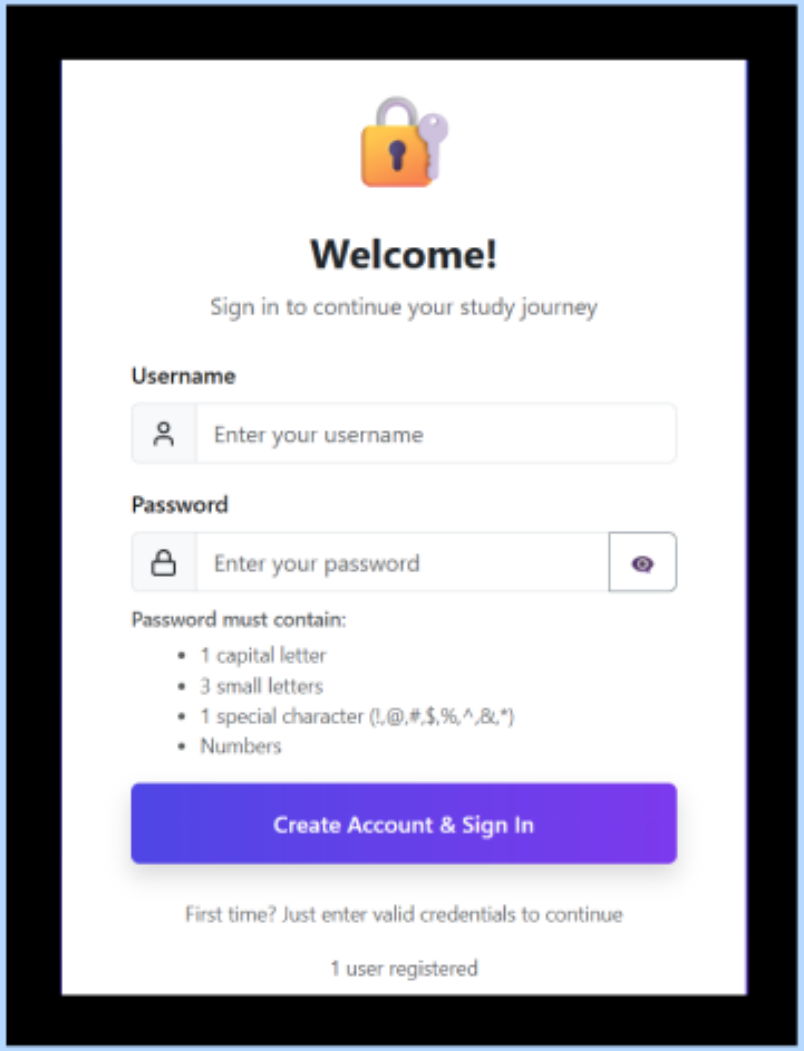
Module 1 – User Authentication & Frontend Basics

Course Concepts:

- Internet & Web Fundamentals: HTTP/HTTPS, Browser basics, Web client-server architecture
- HTML Basics: Document structure, Headings, Paragraphs, Links
- Introductory CSS: Syntax, Selectors, Typography, Colors, Padding/Margin/Border

Project Features:

- Secure Login / Registration
- User roles (Student)
- Simple responsive login page using HTML + CSS
- Password security and validation



KLH (DEEMED TO BE UNIVERSITY)

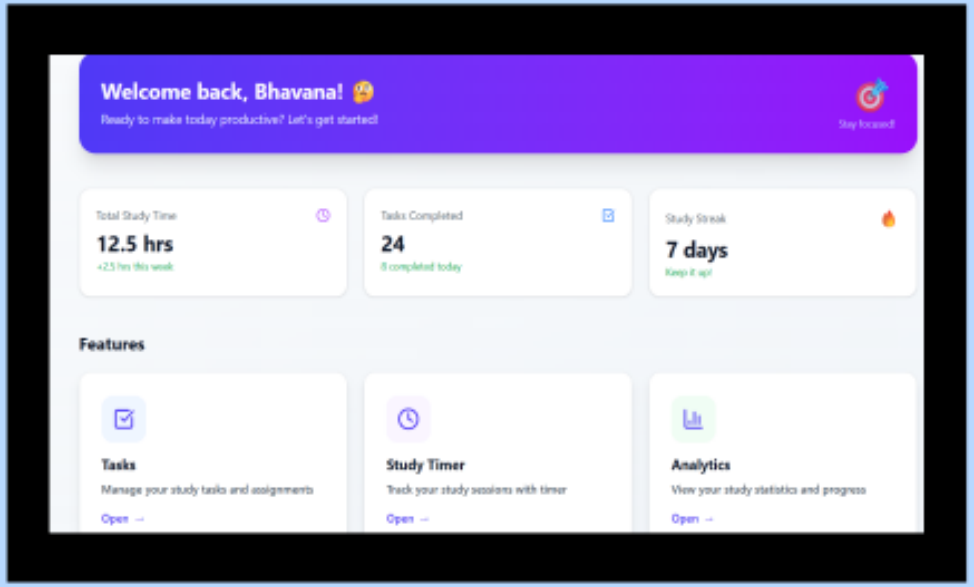
Module 2 – Dashboard and Task Management

Course Concepts:

- HTML Forms & Input Handling: Forms, Input types, Validation
- Semantic Tags & Structured Content: Semantic HTML Tables, Media tags, Accessibility
- Comprehensive CSS Layouts: Flexbox, CSS Grid, Block.
- layout, Media queries, Responsive layout, Bootstrap utilities

Project Features:

- Dashboard showing study statistics
- Add / Edit / Delete tasks
- Categorize tasks by subject
- Responsive, interactive UI with CSS Grid/Flexbox



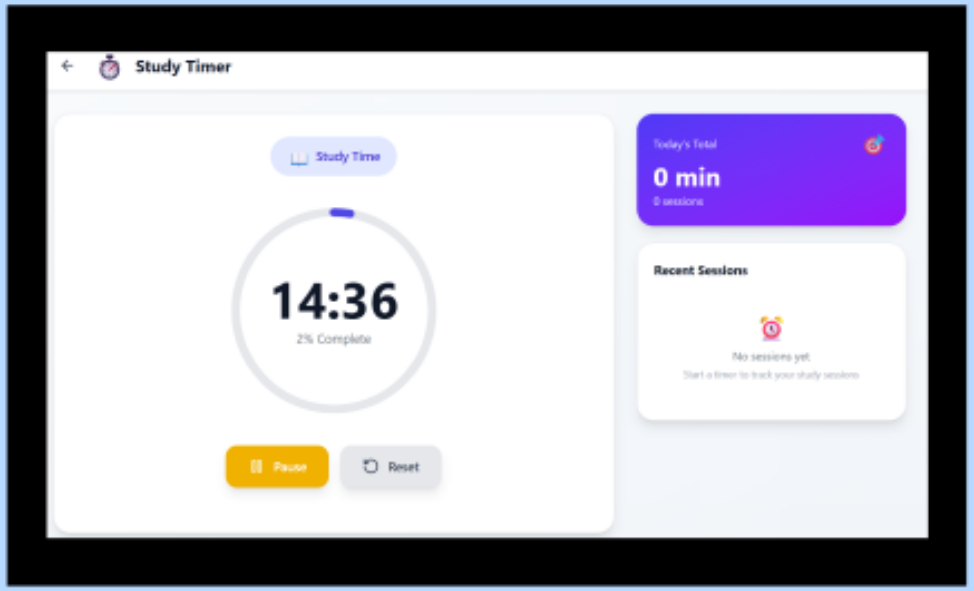
Module 3 – Study Timer & JavaScript Fundamentals

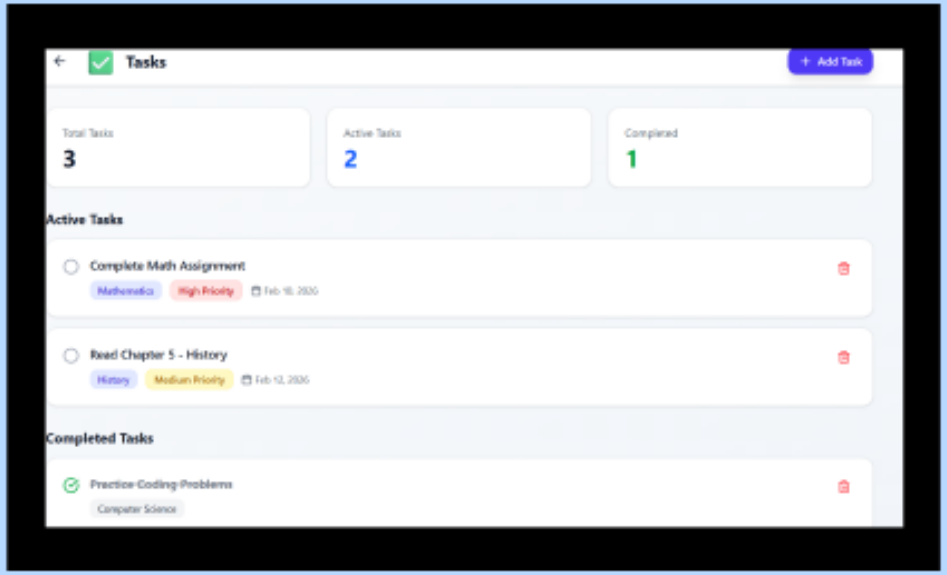
Course concepts:

- JavaScript Essentials: Expressions, Operators, Conditions, Loops, Functions, Arrow functions, Callbacks
- Objects & Arrays: Objects, Arrays, Methods, Object inheritance

Project Features:

- Study Timer to track sessions per task/subject
- Timer data stored in Objects/Arrays
- Start / Pause / Stop timer functionality





Module 4 – Interactivity, DOM & Data Handling

Course Concepts:

- DOM Manipulation & Events: Event handling, DOM updates
- Storage & Async Programming: Local storage, Promises, Async handling

Project Features

- Update task completion status dynamically
- Interactive UI for dashboard graphs
- Store and retrieve study sessions using browser storage

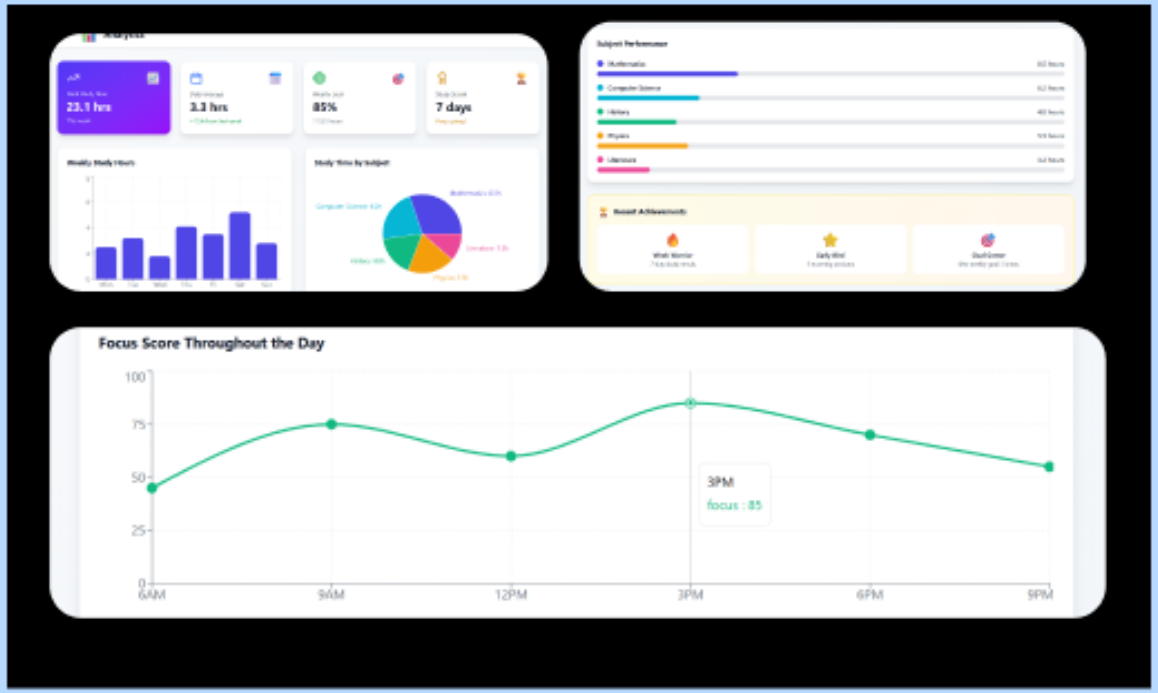
Module 5 – Reports, Analytics & Advanced Web Development

Course Concepts:

- Advanced Web Development: Exception handling, ES6 modules, Form validation, API integration
- Optimization & Deployment: Page load optimization, SEO meta tags, CORS understanding, Deploying the project

Project Features:

- Generate progress reports / charts (Daily / Weekly / Monthly)
- Export reports (PDF optional)
- Deploy project on web server / hosting
- Optimize UI/UX and accessibility compliance



OBJECTIVES

- To develop a system that helps students track study activities efficiently.
- To provide a digital platform for organizing study tasks and subjects
- To monitor daily, weekly, and monthly study hours.
- To improve student productivity through progress analysis.
- To help students build better time management habits.
- To provide visual insights into learning patterns and performance
- To encourage self-monitoring and disciplined study routines.

Course Outcome Attainment

- IMPLEMENTATION OF COURSE OUTCOMES OF FWD IN STUFX

CO1 – Internet Fundamentals, HTML & CSS

HTML document structure and CSS styling concepts were used to design the layout and appearance of the Smart Student Study Tracker web pages.

CO2 – Forms, Semantic Tags & CSS Layout

HTML forms and responsive CSS layouts were implemented to collect and organize user study information efficiently.

CO3 – JavaScript Programming Essentials

JavaScript programming concepts such as functions, arrays and conditions were used to implement application logic.

CO4 – JavaScript Interactivity & DOM

DOM manipulation and event handling were applied to create interactive features in the web application.

CO5 – Advanced Web Development & Deployment

Advanced web development techniques such as form validation, modular scripting and deployment concepts were applied.

Course Outcome Attainment

- IMPLEMENTATION OF COURSE OUTCOMES OF DSA IN STUFX

CO1 – Searching Algorithms

Linear Search is implemented to find tasks in the task list during operations like searching, completing, and deleting tasks.

CO2 – Sorting Algorithms

Tasks are sorted based on deadlines using sorting techniques to organize and manage tasks efficiently.

CO3 – List ADT

ArrayList is used to store and manage study tasks dynamically, allowing easy insertion and traversal of tasks.

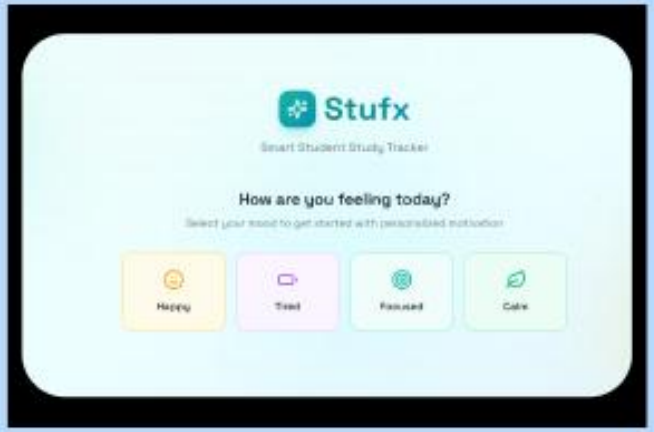
CO4 – Stack ADT

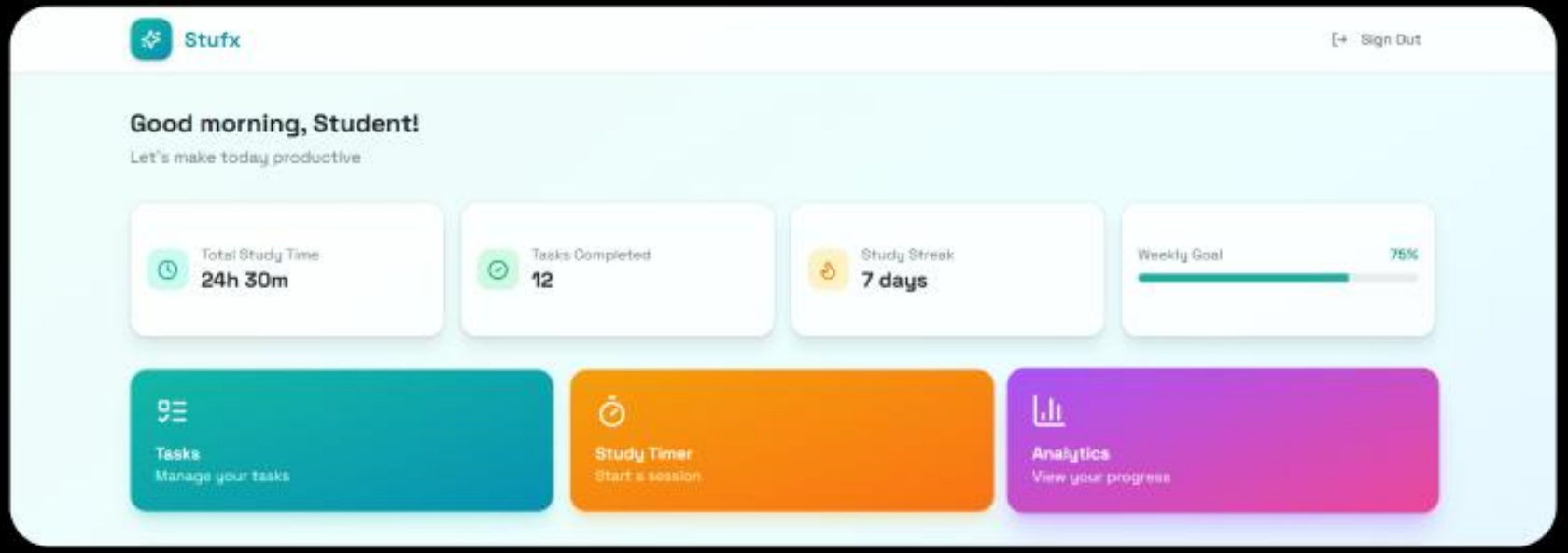
Stack is used to store completed tasks following the Last-In First-Out (LIFO) principle.

CO5 – Queue ADT

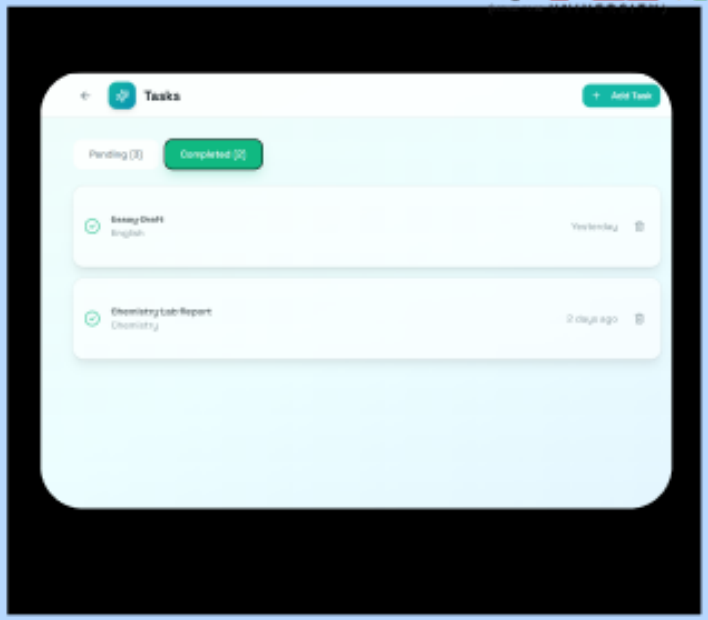
Queue is used to manage pending tasks following the First-In First-Out (FIFO) principle.

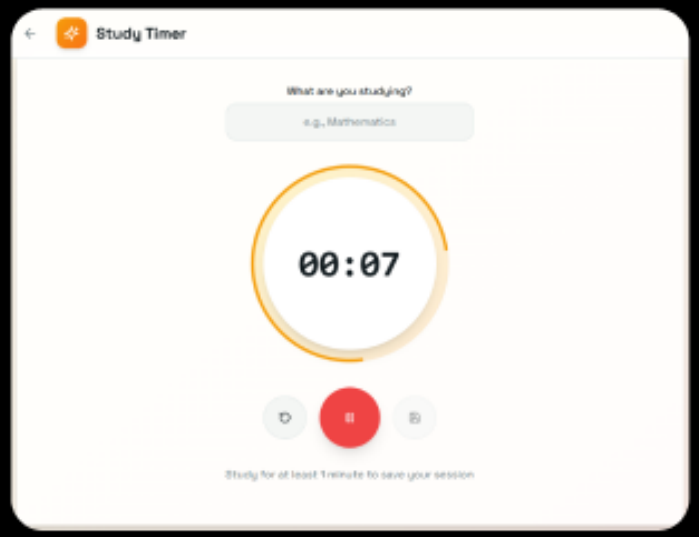
Screenshots for FWD



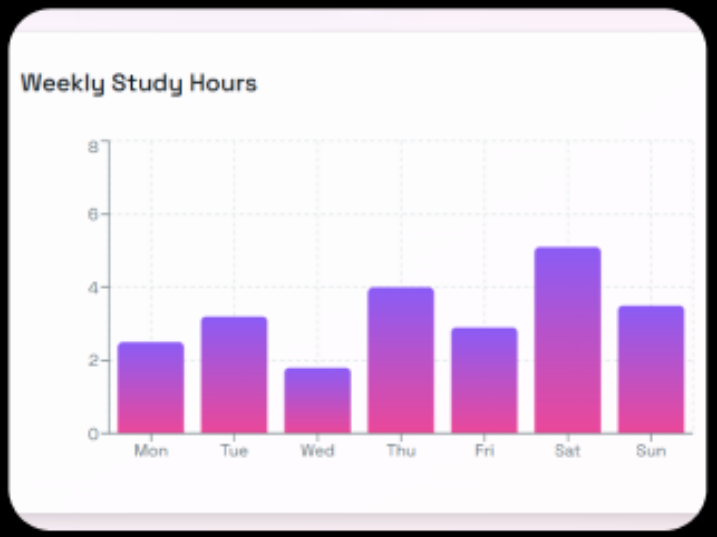


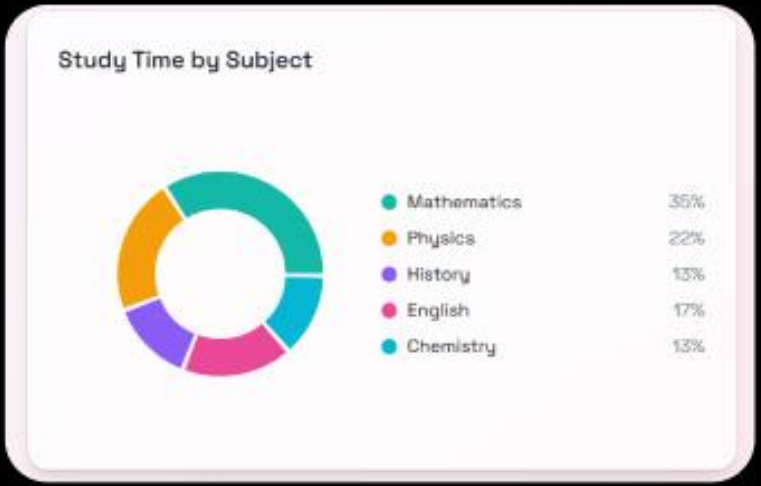
Tasks
Add Task
Pending (3)
Completed (2)
Complete Math Chapter 5
Mathematics
high
Today
Read History Notes
History
medium
Tomorrow
Practice Physics Problems
Physics
low
In 3 days


Tasks

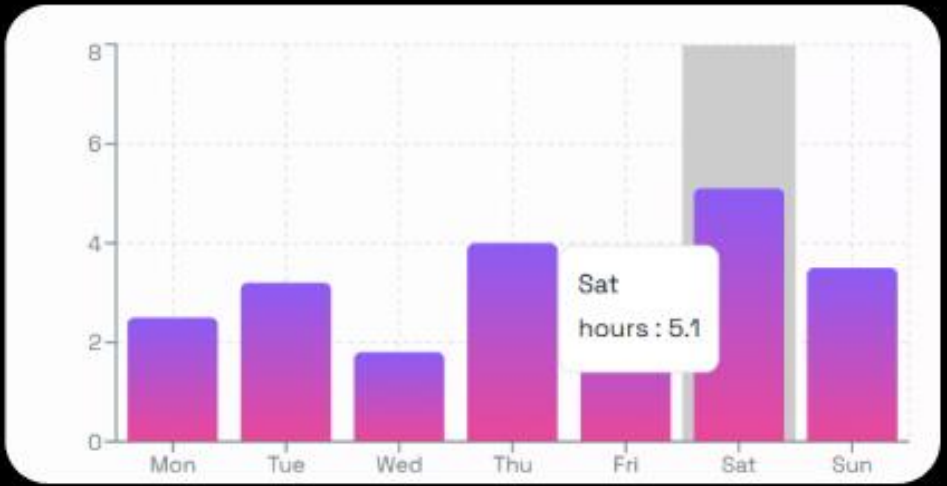

Study Timer
What are you studying?
e.g., Mathematics
00:07
Study for at least 1 minute to save your session

Today's Study Time
1h 45m
Sessions Today
2
Recent Sessions
Mathematics
Today
1h 0m
Physics
Today
45m
History
Yesterday
30m


Weekly Study Hours
8
6
4
2
0
Mon
Tue
Wed
Thu
Fri
Sat
Sun


Study Time by Subject
Mathematics 35%
Physics 22%
History 13%
English 17%
Chemistry 13%

Hours : 4h


8
6
4
2
0
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Sat
hours : 5.1

Focus Score Throughout the Day
100
75
50
25
0
8am
9am
10am
11am
12pm
1pm
2pm
3pm
4pm
5pm
6pm
1pm
Focus Score : 55%
Focus Score

Screenshots for DSA


```java
// ===================== HASHING (search tasks) =====================
class TaskHash {
    HashMap<String, Task> map = new HashMap<>();
    void add(Task t){ map.put(t.name, t);}
    Task search(String name){ return map.getOrDefault(name,defaultValue: null);}
}
```

```java
// ===================== USER =====================
class User {
    String username;
    String password;
    User(String u, String p){ username=u; password=p; }
}
```

```java
// ===================== SEARCHING & SORTING =====================
class SearchingSorting {
    // Quick Sort by priority
    public static void quickSortByPriority(Task[] arr,int low,int high){
        if(low<high){
            int pi = partition(arr,low,high);
            quickSortByPriority(arr,low,pi-1);
            quickSortByPriority(arr,pi+1,high);
        }
    }
    private static int partition(Task[] arr,int low,int high){
        Task pivot = arr[high];
        int i=low-1;
        for(int j=low;j<high;j++){
            if(arr[j].priority > pivot.priority){
                i++;
                Task temp = arr[i]; arr[i]=arr[j]; arr[j]=temp;
            }
        }
        Task temp = arr[i+1]; arr[i+1]=arr[high]; arr[high]=temp;
        return i+1;
    }
}
```

```java
// ===================== STACK (recent completed) =====================
class TaskStack {
    LinkedList<Task> stack = new LinkedList<>();
    void push(Task t){ stack.addFirst(t);}
    Task pop(){ return stack.isEmpty()?null:stack.removeFirst();}
    void printStack(){
        if(stack.isEmpty()){ System.out.println(x: "No recent completed tasks"); return;}
        for(Task t: stack) System.out.println(t.name+" (Priority:"+t.priority+") Deadline:"+t.deadline);
    }
}
```

```java
// ===================== QUEUE (today tasks) =====================
class TaskQueue {
    LinkedList<Task> queue = new LinkedList<>();
    void enqueue(Task t){ queue.addLast(t);}
    Task dequeue(){ return queue.isEmpty()?null:queue.removeFirst();}
    void printQueue(){
        if(queue.isEmpty()){ System.out.println(x: "No tasks for today"); return;}
        for(Task t: queue) System.out.println(t.name+" (Priority:"+t.priority+") Deadline:"+t.deadline);
    }
}
```

```java
// ===================== MAIN PROJECT =====================
public class SmartStudyTrackerDSA {
    static Scanner sc = new Scanner(System.in);
    static ArrayList<User> users = new ArrayList<>();
    static Pattern passwordPattern = Pattern.compile(regex: "^(?=.*[A-Z])(?=.*\\d)(?=.*[!@#$%^&*])[A-Za-z\\d!@#$%^&*]{8,}$");

    // Login / Signup
    static User login(){
        while(true){
            System.out.println(x: "=== Login / Sign Up ===");
            System.out.print(s: "Enter username: "); String uname = sc.nextLine();
            System.out.print(s: "Enter password: "); String pass = sc.nextLine();

            // Check existing
            for(User u: users){
                if(u.username.equals(uname)){
                    if(u.password.equals(pass)){
                        System.out.println("Login successful! Welcome "+uname+"\n");
                        return u;
                    } else{
                        System.out.println(x: "Invalid password! Try again.\n");
                        continue;
                    }
                }
            }
```

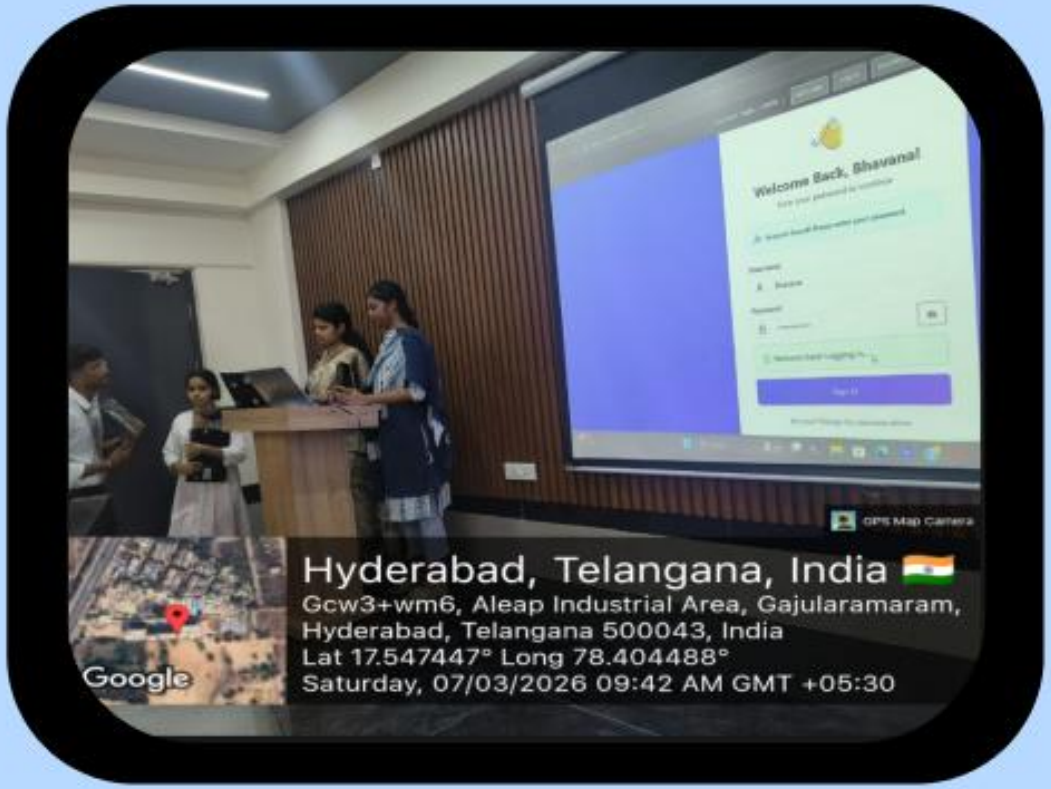
```java
// Dashboard
static void dashboard(){
    TaskStack completedStack = new TaskStack();
    TaskQueue todayQueue = new TaskQueue();
    TaskHeap priorityHeap = new TaskHeap();
    TaskHash taskMap = new TaskHash();
    ArrayList<Task> allTasks = new ArrayList<>();

    while(true){
        System.out.println(x: "\n=== Dashboard ===");
        System.out.println(x: "1. Add Task");
        System.out.println(x: "2. View Today's Tasks");
        System.out.println(x: "3. Mark Task Completed");
        System.out.println(x: "4. View Recent Completed Tasks");
        System.out.println(x: "5. View High Priority Tasks");
        System.out.println(x: "6. Sort Tasks by Priority");
        System.out.println(x: "7. Search Task by Name");
        System.out.println(x: "8. Exit");
        System.out.print(s: "Choice: "); int choice = sc.nextInt(); sc.nextLine();
```

```java
    // New account
    if(!passwordPattern.matcher(pass).matches()){
        System.out.println(x: "Password invalid! Must be 8+ chars, 1 uppercase, 1 number, 1 special char.\n");
        continue;
    }

    User newUser = new User(uname, pass);
    users.add(newUser);
    System.out.println("Account created successfully! Welcome "+uname+"\n");
    return newUser;
}
```

```java
switch(choice){
    case 1:
        System.out.print(s: "Task Name: "); String n=sc.nextLine();
        System.out.print(s: "Priority (1-10): "); int p=sc.nextInt(); sc.nextLine();
        System.out.print(s: "Deadline (YYYY-MM-DD): "); String d=sc.nextLine();
        Task t = new Task(n,p,d);
        allTasks.add(t); todayQueue.enqueue(t); priorityHeap.add(t); taskMap.add(t);
        System.out.println(x: "Task added!");
        break;
    case 2:
        System.out.println(x: "=== Today's Tasks ===");
        todayQueue.printQueue();
        break;
    case 3:
        System.out.print(s: "Task Name to mark completed: "); String cname = sc.nextLine();
        Task ct = taskMap.search(cname);
        if(ct!=null && !ct.completed){
            ct.completed = true; completedStack.push(ct);
            System.out.println(x: "Task marked completed!");
        } else System.out.println(x: "Task not found or already completed!");
        break;
    case 4:
        System.out.println(x: "=== Recent Completed Tasks ===");
        completedStack.printStack();
        break;
    case 5:
        System.out.println(x: "=== High Priority Tasks ===");
        priorityHeap.printHeap();
        break;
```

```java
        break;
    case 6:
        Task[] arr = allTasks.toArray(new Task[0]);
        SearchingSorting.quickSortByPriority(arr,low: 0,arr.length-1);
        System.out.println(x: "=== Tasks Sorted by Priority ===");
        for(Task tt: arr){
            System.out.println(tt.name+" (Priority:"+tt.priority+") Deadline:"+tt.deadline+" Completed:"+tt.completed);
        }
        break;
    case 7:
        System.out.print(s: "Task Name to Search: "); String sname=sc.nextLine();
        Task st = taskMap.search(sname);
        if(st!=null) System.out.println("Found Task: "+st.name+" (Priority:"+st.priority+") Completed:"+st.completed+" Deadline:"+st.deadline);
        else System.out.println(x: "Task not found!");
        break;
    case 8:
        System.out.println(x: "Exiting dashboard. Bye!");
        return;
    default:
        System.out.println(x: "Invalid choice!");
    }
  }
}

Run | Debug
public static void main(String[] args){
    System.out.println(x: "Welcome to Smart Student Study Tracker (DSA Console Version)!");
    User u = login();
    dashboard();
  }
}
```

Geo- Tag Photos



THANK YOU...